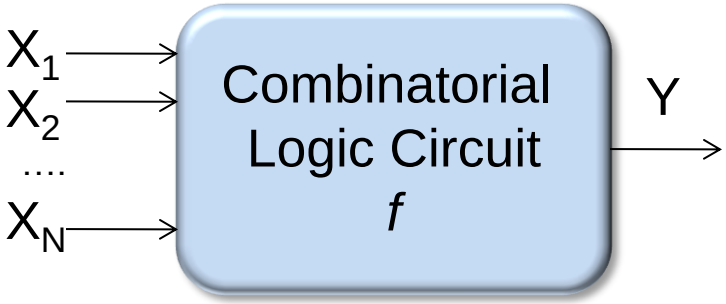
Laboratorio di Elettronica Digitale

- Combinatorial Logic Circuits Basics -

UNIVERSITÀ DEGLI STUDI FIRENZE
DINFO
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Updated 02/2025

- Definitions -



$$Y = f\,(X_1, X_2, \ldots X_N)$$

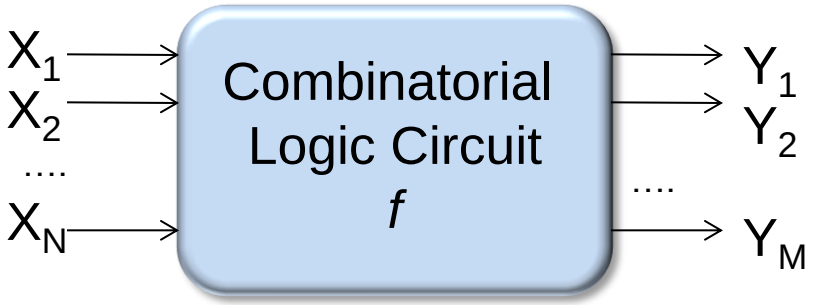
uscita dipende da quello che sta in ingresso

- The <u>combinatorial</u> logic circuit produces the output Y that depends on the input X
- To every input combination $X_{1..N}$ corresponds a specific Y
- The <u>combinatorial</u> circuit is similar to a math function *f* whose domain is the binary space with dimension 'N'.

Nor 'time' neither 'delay' is involved: an input variation produces a simultaneous output change

- Definitions -

- We can have multiple outputs:

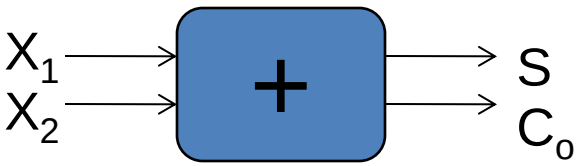


$$Y_1 = f_1 (X_1, X_2, \ldots X_N)$$
$$Y_2 = f_2 (X_1, X_2, \ldots X_N)$$
$$Y_M = f_M (X_1, X_2, \ldots X_N)$$

- What stated for the single output circuit is still valid
- Every output combination $Y_{1..M}$ depends on the input combination $X_{1..N}$

- 3 forms of description -

- Combinatorial circuit example: 2 bit adder:



- The circuit can be described through:

1) Truth table

IN		OUT	
X_1	X_2	S	C_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

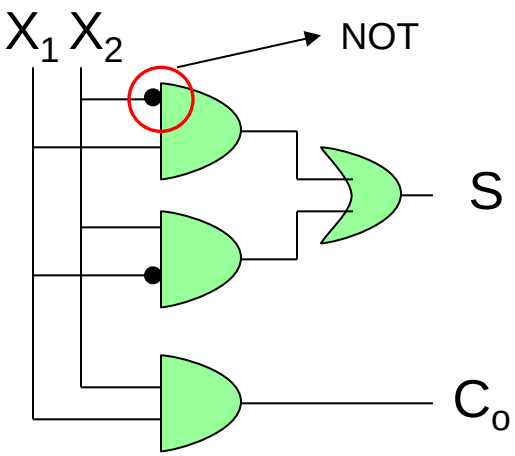
2) Boolean math

$$S = (\overline{X_1} \cdot X_2)+(X_1 \cdot \overline{X_2})$$

$$C_o = X_1 \cdot X_2$$

end fra 2 ingressi

3) Logic Circuit

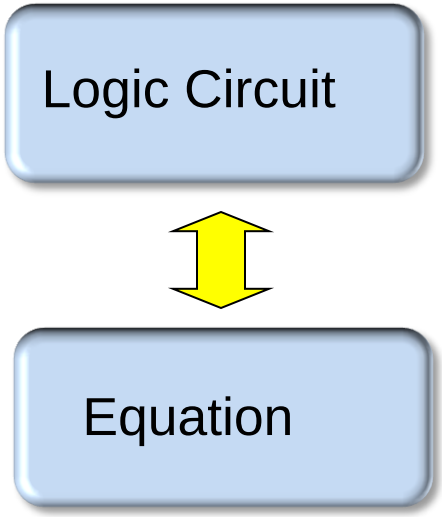


- Relations among the 3 forms -

- The truth table describes the circuit behavior independently to its particular realization

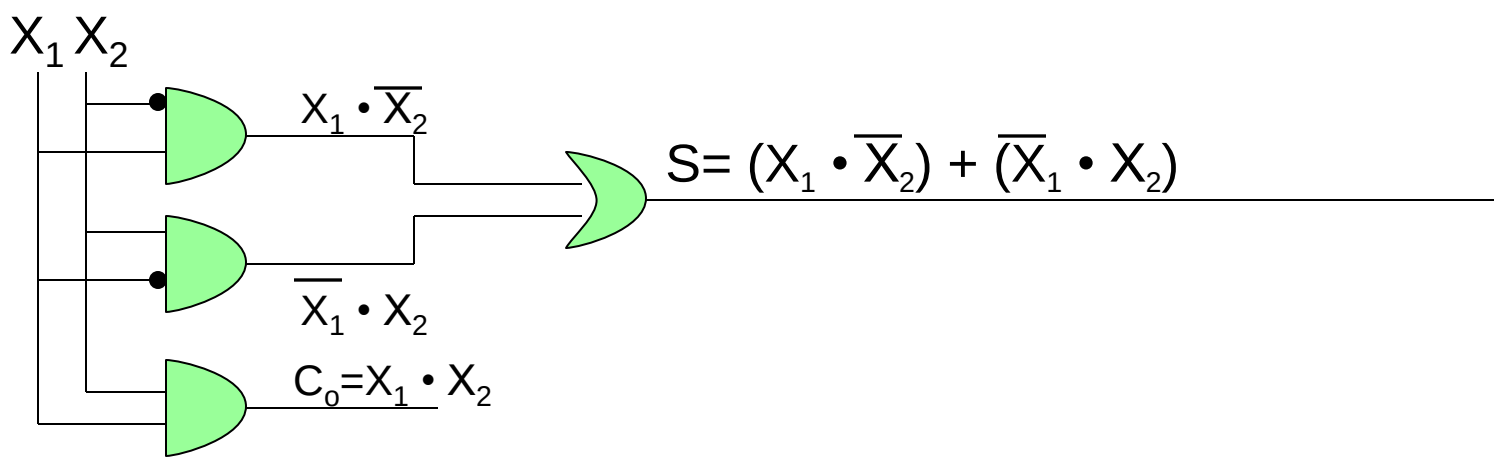
IN		OUT	
X_1	X_2	S	C_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- The Boolean math and logic circuit describe a particular circuit implementation, they are equivalent since we can substitute a gate to each math operation, and vice versa. (for example AND ÷ product; OR ÷ addition)



- Relations among the 3 forms -

- Example of equivalence between Boolean math expression and logic circuit:



- Synthesis and Analysis -

- A circuit with a particular function (Truth table) may be realized in different ways.
- Synthesis consists in designing one or more circuits that implement the given Truth table
- Analysis consists in obtaining the Truth table corresponding to a given logic circuit

IN		OUT	
X_1	X_2	S	C_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Synthesis →

← Analysis

Implementation 1

Logic circuit or Boolean equation

Implementation 2

Logic circuit or Boolean equation

- - - - - -

Implementation N

Logic circuit or Boolean equation

- Synthesis and Analysis -

- Analysis:

  The truth table of a logic circuit with N inputs is obtained by:

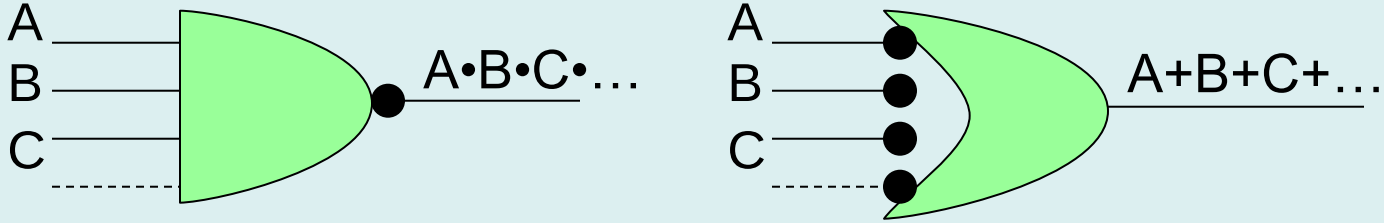
  1. Applying to the input the 2^N binary combinations
  2. Reporting in the truth table the output combination obtained for each input

- Synthesis is a bit more complicated:
  - More circuits are possible that implement the same function
  - Which is the "best" circuit? A criterion is needed. For example
    - Number of gates (input)
    - Input/Output delay
    - Kind of gates (per es. all NAND)
    - Behavior during transitions (see Alee)
    - Power consumption
    - …
  - How can we get the "best" circuit from the truth table?

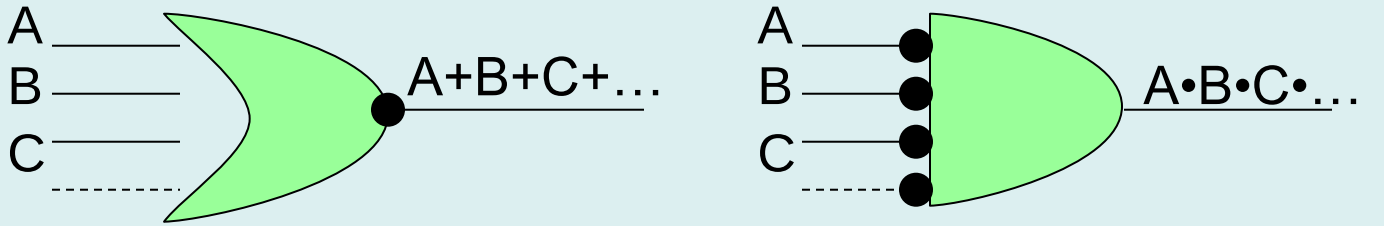
- Algebra of Boolean expressions -

- De Morgan:

$$\overline{A \bullet B \bullet C \bullet \ldots} = \overline{A} + \overline{B} + \overline{C} + \overline{\ldots}$$



$$\overline{A + B + C + \ldots} = \overline{A} \bullet \overline{B} \bullet \overline{C} \bullet \overline{\ldots}$$



- Verification by building the truth tables

- Algebra of Boolean expressions -

- Proprietà distributiva del prodotto:

$$A \bullet (B+C+D\ldots) = A \bullet B + A \bullet C + A \bullet D + A \bullet \ldots$$

- Proprietà distributiva della somma:

$$A + (B \bullet C \bullet D\ldots) = (A+B) \bullet (A+C) \bullet (A+D) \bullet \ldots$$

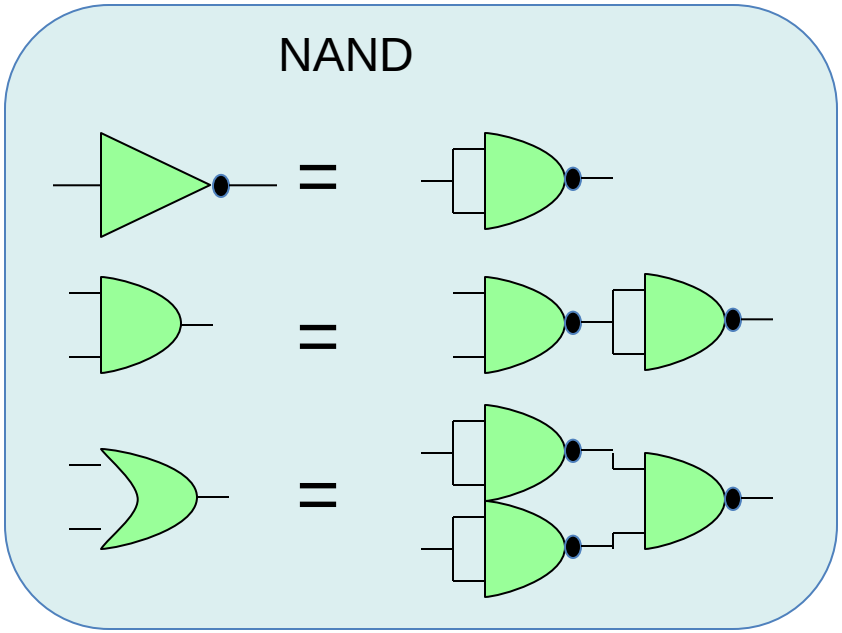
- Verification by building the truth tables

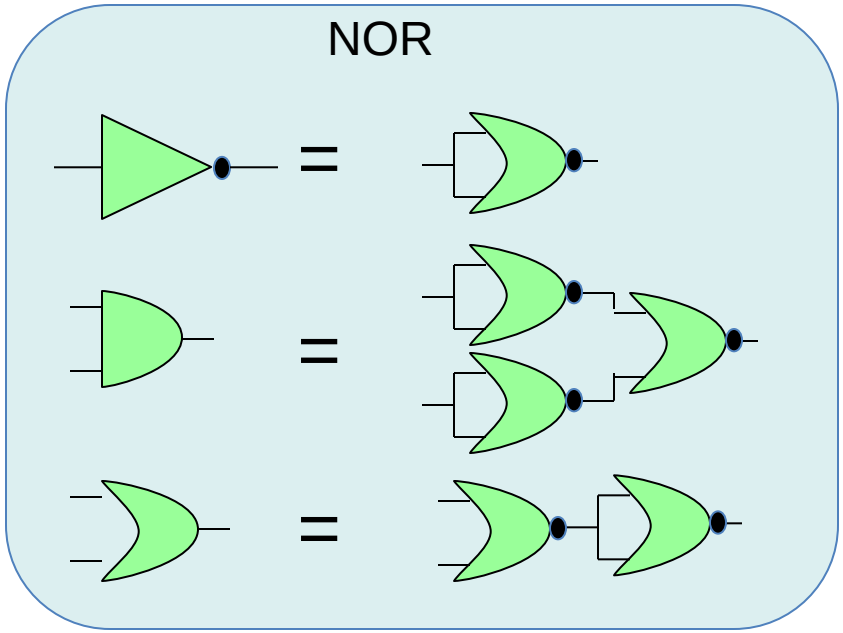
- Universal gates -

Which is the minimal set of gate functions sufficient to build all the possible logic circuits?

For example, a complete set of gates is [NOT, AND, OR].
Can this be reduced?

Yes! : All logic circuits can be obtained from NAND or NOR gates. In fact:

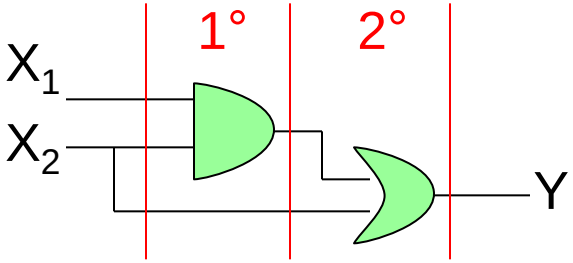




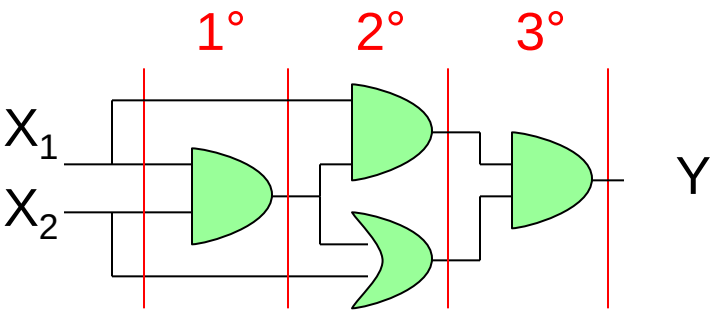
- Number of Levels -

- Number of Levels of a logic circuit:

  It is the number of gates crossed by the longest path from input to output



2 level circuit



3 level circuit

The I/O delay of the circuit is related to the levels number N

If Tp is the delay of a single gate, the delay of the circuit is N*Tp

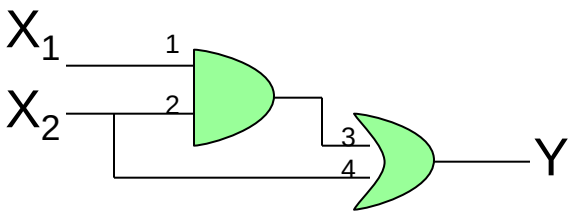
Note: If a circuit includes a gate with several inputs (e.g. AND with 20 inputs) probably such gate is fabricated with several levels

- Complexity -

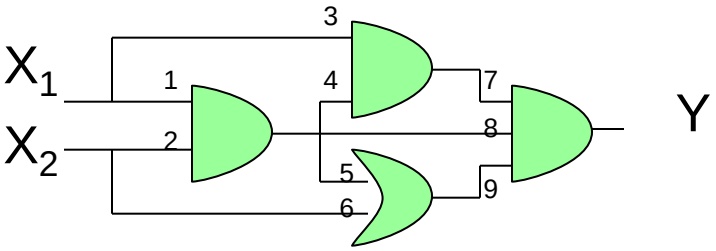
conto ingressi delle porte (complessità approssimata)

- Circuit Complexity:

  It is estimated by the overall number of inputs present in the circuit



Complexity 4



Complexity 9

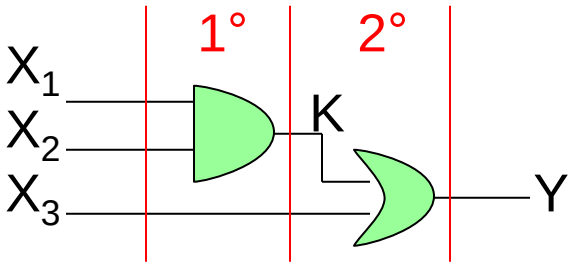
- Alee -

altro possibile criterio

=> fenomeno caratterisco dei circuiti fisici

Different propagation delays along different paths of a circuit can produce glitches on the output of a real circuit.

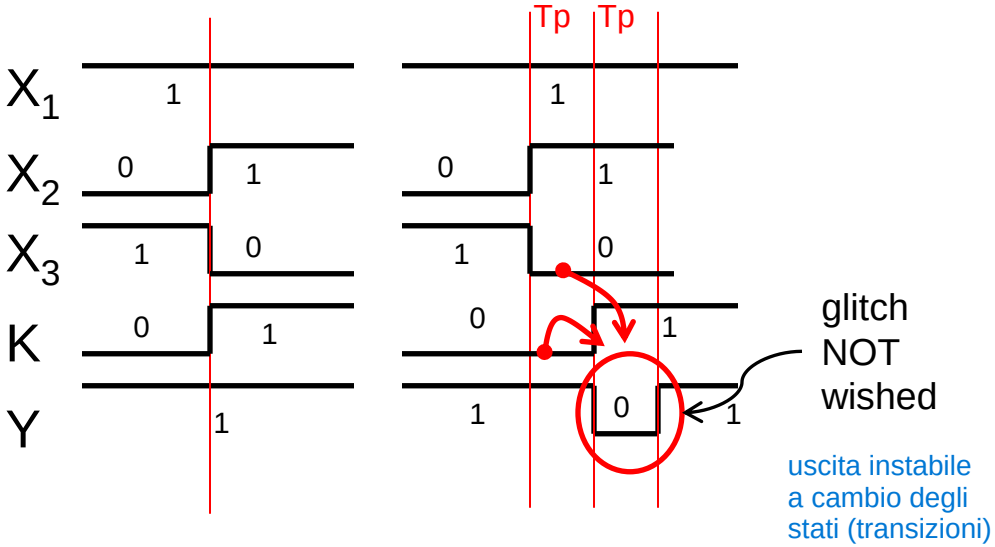
Example:



$Y = X_3 + X_1 \bullet X_2$

Ideal circuit (without dalays)

Real circuit (with delays)



In this example the path X_3-Y has a delay Tp, while X_1 and X_2 are delayed of 2·Tp. This mismatch produces the glitch on Y.

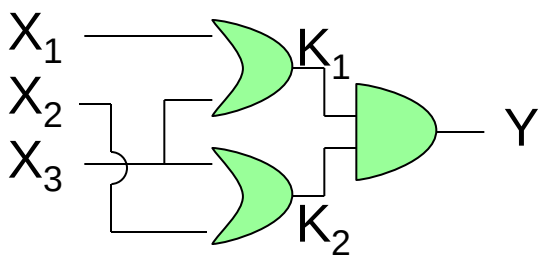
Il criterio in questo caso sarà quindi evitare questo tipo di problemi quando possibile

- Alee -

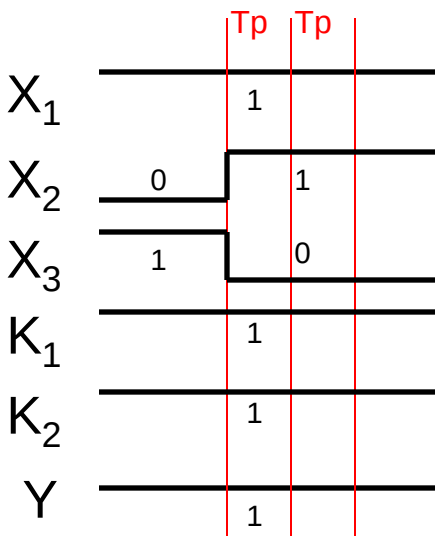
If we use a different circuit that implements the same function :

Proprietà distributiva di "+"

$$Y = X_3 + X_1 \bullet X_2 =$$
$$= (X_3 + X_1) \bullet (X_2 + X_3)$$



idealmente le due porte or funzionano allo stesso



In this case a more complex circuit (6 input instead of 4) solves the glitch

- 2-level synthesis -

- Starting from a Truth table it is always possible to obtain a 2-level circuit by employing AND-OR, OR-AND, NAND-NAND, NOR-NOR gates
  - For example:

IN			OUT
X_1	X_2	X_3	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Synthesis AND-OR:

$$Y = (\overline{X_1} \bullet X_2 \bullet X_3) + (X_1 \bullet \overline{X_2} \bullet \overline{X_3}) + (X_1 \bullet X_2 \bullet \overline{X_3})$$

1. Select the rows with OUT = ‘1’
2. Add a NOT for each low input
3. Each selected row contributes with a AND in the equation
4. All of the AND (rows) are added in a OR

- The equation obtained by this synthesis is called «standard form»

- 2-level synthesis -

Synthesis OR-AND

and fra termini
in or

$$Y = (X_1+X_2+X_3)\bullet(X_1+X_2+\overline{X_3})\bullet(X_1+\overline{X_2}+X_3)\bullet(\overline{X_1}+X_2+\overline{X_3})\bullet(\overline{X_1}+\overline{X_2}+\overline{X_3})$$

IN			OUT
X_1	X_2	X_3	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

1. Select the rows with OUT = ‘0’
2. Add a NOT for each high input
3. Each selected row contributes with a OR in the equation
4. All of the OR (rows) are combined in a AND

- 2-level synthesis -

- Synthesis NAND-NAND

$$Y = \overline{\overline{(\overline{X_1}\bullet X_2\bullet X_3)}\bullet\overline{(X_1\bullet\overline{X_2}\bullet\overline{X_3})}\bullet\overline{(\overline{X_1}\bullet\overline{X_2}\bullet X_3)}}$$

IN			OUT
X_1	X_2	X_3	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

1. Select the rows with OUT = ‘1’
2. Add a NOT for each low input
3. Each selected row contributes with a NAND in the equation
4. All of the NAND (rows) are combined in a NAND

Can be obtained by DeMorgan from AND-OR synthesis

- 2-level synthesis -

- Synthesis NOR-NOR

$$Y = \overline{\overline{(X_1+X_2+X_3)}+\overline{(X_1+X_2+\overline{X_3})}+\overline{(X_1+\overline{X_2}+X_3)}+\overline{(\overline{X_1}+X_2+\overline{X_3})}+\overline{(\overline{X_1}+\overline{X_2}+\overline{X_3})}}$$

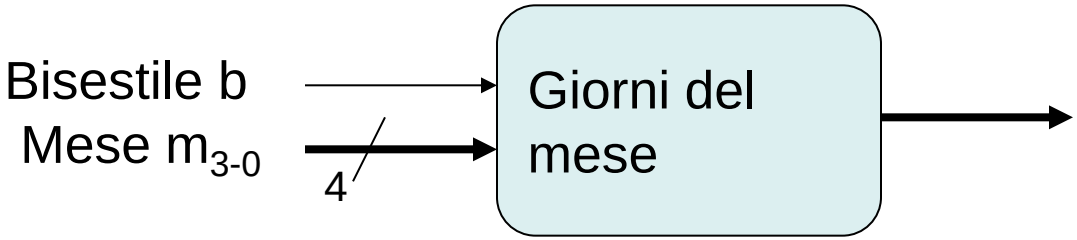
IN			OUT
X_1	X_2	X_3	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

1. Select the rows with OUT = ‘0’
2. Add a NOT for each high input
3. Each selected row contributes with a NOR in the equation
4. All of the NOR (rows) are combined in a NOR

Can be obtained by DeMorgan from OR-AND synthesis

- 2-level synthesis: an example -

- Esempio di sintesi



- Ingressi:
  - Anno bisestile o non bisestile (codificato su di un bit)
  - Mese corrente (codificato su 4 bit)

- Uscita:

  Numero giorni del mese (28,29,30,31), codificato su 2 bit

- 2-level synthesis: an example -

- Tabella di verità:

	m_3	m_2	m_1	m_0	b	g_1	g_0
-	0	0	0	0	-	-	-
gen	0	0	0	1	-	1	1
feb	0	0	1	0	0	0	0
feb	0	0	1	0	1	0	1
mar	0	0	1	1	-	1	1
apr	0	1	0	0	-	1	0
mag	0	1	0	1	-	1	1
giu	0	1	1	0	-	1	0
lug	0	1	1	1	-	1	1
ago	1	0	0	0	-	1	1
set	1	0	0	1	-	1	0
ott	1	0	1	0	-	1	1
nov	1	0	1	1	-	1	0
dic	1	1	0	0	-	1	1
-	1	1	0	1	-	-	-
-	1	1	1	-	-	-	-

La rete ha 5 ingressi, la tabella dovrebbe specificare le uscite per 32 combinazioni

Con il simbolo “-” si indica “don’t care”, semplificando cosi’ la tabella

$$g_1=(m_3+m_2+\overline{m_1}+m_0+b)\bullet(m_3+m_2+\overline{m_1}+m_0+\overline{b})=$$
$$=(m_3+m_2+\overline{m_1}+m_0)$$

$$g_0=(m_3+m_2+\overline{m_1}+m_0+b)\bullet(m_3+\overline{m_2}+m_1+m_0)\bullet$$
$$\bullet(m_3+\overline{m_2}+\overline{m_1}+m_0)\bullet(\overline{m_3}+m_2+m_1+\overline{m_0})\bullet$$
$$\bullet(\overline{m_3}+m_2+\overline{m_1}+\overline{m_0})$$

- 2-level synthesis: an example -

Karnough map

g_0 b=0		m_3 / m_2			
		00	01	11	10
m_1 m_0	00	-	0	1	1
	01	1	1	-	0
	11	1	1	-	0
	10	0	0	-	1

g_0 b=1		m_3 / m_2			
		00	01	11	10
m_1 m_0	00	-	0	1	1
	01	1	1	-	0
	11	1	1	-	0
	10	1	0	-	1

$$g_0 = \overline{m_3} \bullet m_0 + m_3 \bullet \overline{m_0}$$

$$g_0 = (\overline{m_3} + \overline{m_0}) \bullet (m_3 + \overline{m_2} + m_0)$$

In definitiva:

$$g_0 = \overline{b} \bullet (\overline{m_3} \bullet m_0 + m_3 \bullet \overline{m_0}) + b \bullet (\overline{m_3} + \overline{m_0}) \bullet (m_3 + \overline{m_2} + m_0)$$

- 2-level synthesis: an example -

- Oppure, considerando una singola mappa tridimensionale:

g_0 b=0		m_3 / m_2			
		00	01	11	10
m_1 m_0	00	-	0	1	1
	01	1	1	-	0
	11	1	1	-	0
	10	0	0	-	1

g_0 b=1					
m_1 m_0	11	1	1		
	10	1	0	-	1

Unica cella che cambia

$$g_0 = (\overline{m_3} \bullet m_0 + m_3 \bullet \overline{m_0}) + b \bullet \overline{m_3} \bullet \overline{m_2}$$

- 2-level synthesis: summary -

- 2-level synthesis is <u>always</u> possible, but not always the most convenient
- The OR-AND and AND-OR are called ‘standard representations’
- The AND-OR / NAND-NAND synthesis is more advisable when the Truth Table has less ‘1’ than ‘0’ in the out column
- The OR-AND / NOR-NOR synthesis is more advisable when the Truth Table has less ‘0’ than ‘1’ in the out column
- It is easily produced by a software/computer

- What should we remember -

- A combinatorial circuit produces immediately (ideally no delay) an output combination for each input combination
- A logic function has 3 possible descriptions: Truth Table, Equation, Logic Circuit
- We have several logic «bricks» like OR, AND, NOT, XOR, NAND, etc, but all logics can be obtained by using only NOR or NAND gates
- Every Truth Table can be synthetized in a lot of different ways, and in particular it can be always synthetized in the standard form (2-level AND/OR)
- Different realizations of the same Truth Table differ for complexity, number of levels, alee, kind of gates, etc